Machine-Learning-Assisted Adaptive CPU Scheduling

Predicting workload behavior. Optimizing scheduling decisions.

Ujval Sai

Individual project · [Course name and code] · [Instructor] · [University] · [Submission date]

Repository: https://github.com/ujvall/schedX · Reproducibility reference: commit `e848cf2` (https://github.com/ujvall/schedX/commit/e848cf2)

***Abstract*—Round Robin (RR) scheduling depends on a time quantum, and there is no single value that suits everything. A small quantum gets jobs started quickly and helps short ones, while a large quantum cuts down on context switches and works well when jobs are about the same length. This case study looks at whether predicting how a workload behaves can pick a better quantum than a fixed setting. To test this, SchedX combines a repeatable discrete-event simulator, five classical scheduling algorithms, a synthetic workload generator, and machine-learning (ML) controllers that choose a quantum using only what is known at decision time. The controllers were chosen on validation data and frozen before testing. The main experiment ran them at three operating points of 6, 14, and 22 processes per workload, with 1,000 independently generated workloads at each. Compared with the best fixed quantum (RR1), the frozen controller lowered per-workload mean turnaround by 0.06%, 0.30%, and 0.49% (geometric means). These gains hold up after Holm correction, but all of them fall short of the 1% practical threshold set in advance, and mean response time gets worse. A censoring-aware estimator gave better online estimates but no better decisions. Everything here comes from a CPU-only simulator running synthetic workloads; nothing was measured on a real kernel or on hardware.**

***Index Terms*—CPU scheduling, Kaplan–Meier estimator, machine learning, paired statistical testing, Round Robin, simulation, time quantum**

I. Problem Statement and Objectives

Most operating systems share the CPU by giving each job a slice of time, and the length of that slice is a compromise. Make it very small and jobs share the CPU almost evenly, but the system spends a lot of its effort switching between them. Make it very large and RR ends up behaving like first-come, first-served (FCFS) [1]. Since no fixed quantum is best for every workload, it is natural to ask whether a scheduler that *predicts* workload behaviour could choose better.

Research question. Can predicting process behaviour lead to better scheduling-policy or time-quantum decisions than fixed, rule-based CPU scheduling?

Objectives.

1) Build a repeatable simulator with the classical algorithms and one shared set of metrics.
2) Generate labelled synthetic workloads and test whether their behaviour can be predicted from information a scheduler would really have.
3) Use those predictions to choose an RR quantum, then compare the result fairly against fixed quanta.
4) Test the frozen controllers at three independent workload sizes.

II. Background and Related Work

If job lengths are known in advance, Shortest Remaining Time First (SRTF) gives the lowest possible mean turnaround on a single processor [3]. A real scheduler doesn't know how long the next CPU burst will be, which is why policies like RR that don't need this knowledge are used in practice. How the quantum affects delay has been studied for a long time through queueing analyses of time-shared systems [2], and textbooks describe the same trade-off between response time and switching overhead [1]. Machine learning has also been tried on this kind of problem, including for Linux process scheduling [12]. This project doesn't propose a new scheduling algorithm. What it adds is a controlled test, with safeguards against data leakage, of whether learned quantum selection actually helps, and it reports the result even though it is mostly negative.

III. SchedX Design

A. Simulator and Algorithms

The simulator is a deterministic discrete-event model of a single CPU, with time counted in whole ticks. It implements FCFS, non-preemptive Shortest Job First (SJF), SRTF, RR, and Priority scheduling (both non-preemptive and preemptive), and its tie-breaking rules are documented. An adaptive version of RR lets the quantum change during a run. A slice that has already started keeps its quantum, and with a constant quantum this version matches ordinary RR exactly, which the tests confirm. Context switches are counted but take no simulated time.

Metrics. For each job:

- turnaround = completion − release;
- waiting = turnaround − CPU time;
- response = first dispatch − release.

Throughput and CPU utilization are measured from the first arrival to the last completion. A context switch is counted whenever a different job is dispatched.

B. From Workloads to Jobs

Each generated process alternates between CPU bursts and I/O waits. For scheduling, every CPU burst becomes its own job, released at the moment it would start if nothing else were competing for the CPU (we call this the *burst-trace projection*). I/O is recorded but never blocks anything, so this is a CPU-only, open-loop model.

C. Prediction and Control

- **Workload classifiers.** Logistic Regression, Decision Tree, and Random Forest models (scikit-learn [4]; Random Forest

[5]) predict whether a workload is CPU-bound, I/O-bound, or mixed.

- **Quantum controller.** The scheduler starts on RR1, the best fixed quantum on the training and validation data. After a set number of jobs have finished, it builds features from the schedule that has actually run so far, decides once whether to stay on RR1 or move to a quantum in {2, 4, 8}, and carries on. If a workload ends before that point, it simply stays on RR1. A whitelist blocks any feature that wouldn't be available at decision time.
- **Censoring-aware estimator.** Under RR1 the short jobs finish first, so the completed bursts are not a fair sample of all bursts. The estimator treats each unfinished job as *right-censored*: all we know is that its burst is longer than the CPU time it has had so far. It applies the Kaplan–Meier estimator [6] and, rather than guessing at what it can't see, reports how much of the distribution is still unknown.

IV. Data and Methodology

A. Workloads and Labels

A seeded generator builds each workload independently from a (master seed, index) pair, using:

- process counts of 4–24;
- exponential gaps between arrivals;
- a mix of CPU-heavy and I/O-heavy processes;
- Gamma-distributed integer bursts, with variability that differs per workload.

Labels come from the traces that were actually generated, not from what the generator was aiming for. A process is I/O-bound if more than half its time is I/O. A workload is CPU-bound if at most 25% of its processes are I/O-bound, I/O-bound if at least 75% are, and mixed otherwise. The development dataset has 1,500 workloads (21,453 processes; 484 CPU-bound, 527 I/O-bound, 489 mixed), split by workload into 900 training, 300 validation, and 300 test workloads, with class proportions kept equal across the splits.

B. Information Available to the Scheduler

TABLE I
Feature availability categories.

Category	Content	Used by controller
A: submission	process count, arrival times, priorities	yes
O: online observation	released jobs, CPU received so far, completed burst lengths, switches	yes
E: censoring-aware	Kaplan–Meier estimates of unfinished bursts	yes (estimator variant)
B: first *k* bursts (Phase 2)	optimistic: includes processes that have not arrived yet	classifiers only
C: retrospective	lifetime totals, labels	never

C. Evaluation Protocol

Models were trained on training data and chosen on validation data, and each controller was committed to Git before any test run. All comparisons are *paired*: every method schedules exactly the same job list for a given workload, and the main experiment stores a hash of each method's input so this can be checked. The main outcome is each workload's mean job turnaround, `T`.

We report it in two ways:

- **Geometric-mean change** (the measure fixed in advance): the geometric mean over workloads of `T_method / T_baseline`, minus 1. Every workload counts equally, whatever its size.
- **Aggregate change:** the change in mean turnaround across all workloads, which gives long-running workloads more weight.

Significance comes from two-sided paired sign-flip permutation tests of the mean log ratio [8], and confidence intervals from a paired percentile bootstrap with 10,000 resamples [7]. Holm's procedure [9] corrects for multiple comparisons within families of hypotheses chosen in advance. We only call a result **practically meaningful** if the whole 95% CI lies below −1%. That threshold was set before the experiment, at about a third of what an oracle could gain (Section V-B).

D. Main Experiment: Three Operating Points

Three new sets of workloads were generated under the same rules, changing only the number of processes per workload. All three sizes are inside the range the controllers were trained on, so no extrapolation is involved.

TABLE II
Operating points in the main experiment.

Operating point	Processes per workload	Workloads	Master seed	Total jobs
Small	6	1,000	5101	32,326
Medium	14	1,000	5102	77,567
Large	22	1,000	5103	121,900

None of the 3,000 workloads matches any of the 2,100 used earlier, based on hashes of their full input traces.

V. Results

A. Development Results

Classification (Phase 2). On the 300 test workloads, Logistic Regression (chosen by validation score) reached 87.6% accuracy (macro-F1 0.874), the Decision Tree 86.4%, and the Random Forest 89.2%. Even so, using the predicted class to pick a policy did not reliably beat a fixed RR2: every 95% CI for the regret difference included zero.

Quantum controller (Phase 3). On the same 300 test workloads, the controller changed turnaround by −0.38% (95% CI −0.75% to −0.04%) compared with RR1, yet the Wilcoxon signed-rank test we had planned to use, with Pratt's handling of zeros [10, 11], gave $p = 0.50$. A later audit explained the mismatch. The bootstrap describes the *mean* log ratio, whereas the rank test, with 275 of 300 pairs tied, behaves almost like a sign test. We therefore treated this result as inconclusive.

Estimator (Phase 4). On training data, the Kaplan–Meier estimator cut the error in the estimated restricted mean burst from 0.43–0.50 to 0.08–0.10 (mean absolute log error), but better estimates did not lead to better decisions. Against the Phase 3 controller, the estimator-based controller changed turnaround by +0.64% (95% CI +0.02% to +1.28%, Holm p = 0.14) on the established test split and by +0.08% (95% CI −0.35% to +0.51%) on 600 fresh workloads.

B. Main Experiment

TABLE III
CHANGE IN TURNAROUND COMPARED WITH RR1 (NEGATIVE = FASTER; 95% CIS IN BRACKETS; HOLM CORRECTION OVER SIX PRIMARY HYPOTHESES).

Size	Controller	Geometric-mean change	Aggregate change	Holm p	Classification
6	Phase 3	−0.06% [−0.10, −0.02]	−0.01%	0.026	supported, below 1%
14	Phase 3	−0.30% [−0.53, −0.08]	−0.12%	0.038	supported, below 1%
22	Phase 3	−0.49% [−0.72, −0.26]	−0.28%	0.0003	supported, below 1%
6	Phase 4	+0.13% [−0.19, +0.45]	−0.04%	1.00	negligible
14	Phase 4	−0.04% [−0.38, +0.30]	+0.12%	1.00	negligible
22	Phase 4	−0.10% [−0.45, +0.24]	−0.02%	1.00	negligible

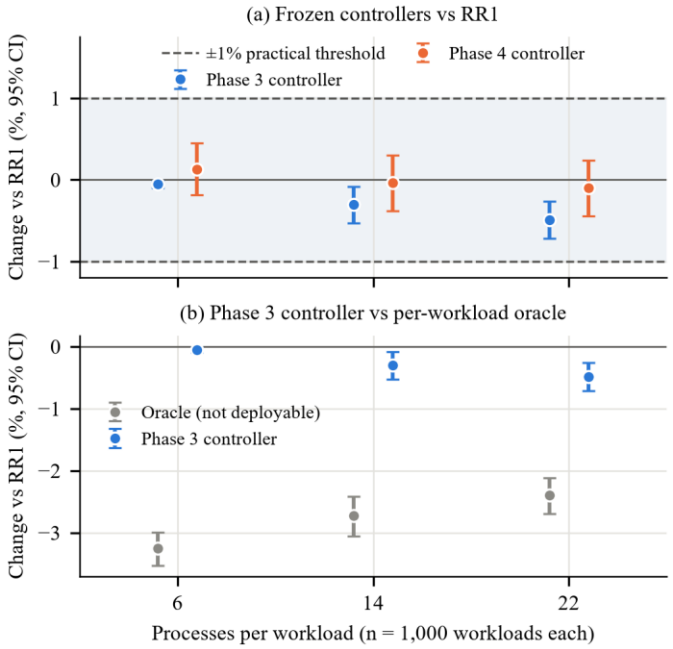


Fig. 1. (a) Geometric-mean change in per-workload turnaround compared with RR1, with 95% paired bootstrap CIs; dashed lines mark the ±1% practical threshold. (b) The Phase 3 controller alongside the per-workload oracle, which could not be deployed. Drawn from reports/phase5/results.json by docs/case_study_build/report_figures.py.

The Phase 3 controller beat RR1 at every size, and the gain grew as the number of processes went up. At 22 processes, the aggregate change of −0.28% means mean turnaround fell from 305.70 to 304.85 ticks. The controller rarely acted, changing the quantum on just 3.2%, 7.7%, and 10.3% of workloads. At 6 processes, 13.2% of workloads finished before the decision point and stayed on RR1. An *oracle* that knows how every fixed quantum would turn out on each workload would gain 3.25%, 2.73%, and 2.40%, so most of the possible improvement is still left on the table. The Phase 4 controller switched far more often (29–42% of workloads) with no measurable turnaround benefit.

C. Secondary Outcomes

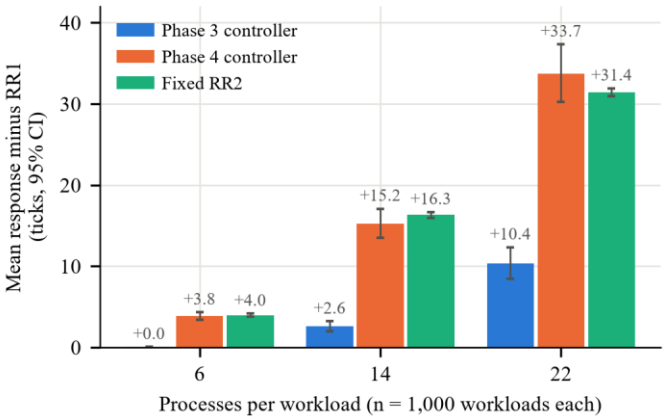


Fig. 2. Increase in mean response time compared with RR1 (ticks, 95% paired bootstrap CIs) at each operating point. Same source and script as Fig. 1.

- **Response time.** The Phase 3 controller raised mean response time compared with RR1 by +0.03 ticks (6 processes), +2.57 ticks [1.98, 3.19] (14), and +10.36 ticks [8.42, 12.32] (22). This makes sense, since a larger quantum means waiting jobs get their first turn later. The Phase 4 controller's increases (+3.8, +15.2, +33.7 ticks) were close to those of a fixed RR2.
- **Context switches.** The Phase 3 controller reduced these slightly (for example, from 891.1 to 848.1 per workload at 22 processes).
- **Throughput and CPU utilization.** These did not change between methods at any operating point (for example, 0.150 jobs per tick and 98.7% utilization at 22 processes). That is expected, because every policy keeps the CPU busy whenever work is waiting and all of them receive the same jobs.

VI. DISCUSSION

Statistical versus practical significance. With 1,000 paired workloads per size, the experiment can pick up very small effects. The Phase 3 benefit is statistically real, shows up at all three sizes, and appeared again on new data, but it stays under half a percent. Pooled with equal weight across the sizes, it is −0.28% (95% CI −0.39% to −0.17%). The response-time cost is larger in relative terms, so someone running a real system could reasonably prefer to stay on a fixed RR1.

Why the gains are small. On training data, the true variability of job lengths was strongly related to whether a large quantum beats RR1 (Spearman 0.79), but the variability *observed* at decision time was only moderately related (0.46). With a small default quantum, long jobs stay hidden until late in the run, so the controller mostly sees short bursts and rarely switches. The censoring-aware estimator fixed the bias in the part of the distribution it can see, but it could not recover the

long tail it can't see, and the extra switches it caused won about as often as they lost.

Why a negative result still matters. A weak signal is easy to overstate, for example by using features that wouldn't exist at decision time (category B) or by tuning on test data. Frozen controllers, paired inputs, statistics fixed in advance, and independent operating points guard against both, so the conclusion can be trusted within its limits: in this simulator, learned quantum selection gives at most a small turnaround gain, and it costs response time.

VII. Limitations and Threats to Validity

1) **Synthetic workloads.** Everything comes from one family of generated workloads, with no real process traces.
2) **Simulator only.** The model is CPU-only and open-loop: I/O doesn't block, and contention doesn't delay later bursts. Context switches are free here, which flatters RR1 because it switches so often. **No kernel-level or hardware measurement was performed.**
3) **One decision per workload.** The controller decides only once, and the default quantum limits what it can see before then.
4) **The 1% threshold** is a judgement call made in advance. A different threshold would change the "practical" label but not the estimates.
5) **Test reuse.** The Phase 3 test split was used again in Phase 4. The fresh workloads in Phase 4 and the independent operating points in Phase 5 address this.
6) **One test environment.** Reproduction was verified only on Windows with Python 3.11.9; other platforms and library versions are untested.

VIII. Conclusion

SchedX shows that ML-assisted quantum selection can be tested rigorously, and that in this simulated setting the benefit is small. The frozen controller improved mean turnaround over the best fixed quantum by 0.06–0.49% across three workload sizes. That improvement is statistically supported but below the 1% practical threshold, and it comes at the cost of response time. A better estimate of unfinished work did not produce better decisions. Useful next steps would be to model the cost of context switches and I/O blocking, allow several decisions during a run, and choose controllers by a criterion that also takes response time into account.

IX. Individual Contribution and AI-Assistance Disclosure

Contribution. This is an individual project by Ujval Sai (GitHub `ujvall`). The repository has one Git author (`cremebits`, the author's Git identity), and the author directed every phase of the project.

AI assistance. The SchedX code, tests, analysis scripts, and documentation were developed with substantial help from an AI coding assistant (Claude), working under the author's direction. This report was also written with that assistance, based on the committed evidence. All reported numbers come from the committed result files; none were produced by hand. [Add the review and work you did yourself, and any wording your course requires.]

X. Reproducibility

The code, configurations, seed manifests, and raw per-workload CSVs are all in the repository, under `src/schedx/`, `configs/`, `scripts/`, and `reports/phase2–phase5/`. Phase 5's raw inputs are in `reports/phase5/inputs/`, and its per-workload outcomes are in `raw_workload_results.csv` in `reports/phase5/`. To reproduce the results with Python 3.11, starting from a fresh clone:

```
pip install -e ".[dev]"
python scripts/generate_dataset.py
python scripts/train_evaluate.py
python scripts/phase3_train.py
python scripts/phase4_train.py
python scripts/phase5_run.py
python scripts/phase5_audit.py
python -m pytest -q
```

A final audit (Phase 6) regenerated all Phase 3–5 evidence identically from a fresh clone. Phase 2 also matched, apart from wall-clock timing fields.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*. Wiley, 2018. ISBN 978-1-119-32091-3.

[2] L. Kleinrock, "Time-shared systems," *Journal of the ACM*, vol. 14, no. 2, pp. 242–261, 1967. doi: 10.1145/321386.321388.

[3] L. Schrage, "A proof of the optimality of the shortest remaining processing time discipline," *Operations Research*, vol. 16, no. 3, pp. 687–690, 1968. doi: 10.1287/opre.16.3.687.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[6] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *Journal of the American Statistical Association*, vol. 53, no. 282, pp. 457–481, 1958. doi: 10.1080/01621459.1958.10501452.

[7] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. doi: 10.1214/aos/1176344552.

[8] P. Good, *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 2005. doi: 10.1007/b138696.

[9] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

[10] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945. doi: 10.2307/3001968.

[11] J. W. Pratt, "Remarks on zeros and ties in the Wilcoxon signed rank procedures," *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959. doi: 10.1080/01621459.1959.10501526.

[12] A. Negi and P. Kumar, "Applying machine learning techniques to improve Linux process scheduling," in *TENCON 2005 – IEEE Region 10 Conference*, 2005, pp. 1–6, doi: 10.1109/TENCON.2005.300837.